Git & Docker Technical Assignment

Question:

Online Course Learning Management System: Develop an e-learning platform where instructors can upload courses and learning materials while students can enroll, attend assessments, track progress, and receive certificates.

1. Problem Analysis

Problem Statement

The **Online Course Learning Management System (LMS)** is an e-learning platform that allows instructors to create and upload courses and learning materials. Students can register, enroll in courses, access learning materials, attend assessments, track their learning progress, and receive certificates after successfully completing the required course activities.

Objectives

- To provide an online platform for course management.
- To allow instructors to upload courses and learning materials.
- To allow students to enroll in available courses.
- To conduct online assessments.
- To track student learning progress.
- To generate certificates for completed courses.
- To maintain course, student, assessment, and progress information securely.

Functional Requirements

Instructor:

- Login/Register
- Create courses
- Upload course materials
- Update/delete courses
- Create assessments
- View student enrollment and progress

Student:

- Register/Login
- Browse available courses
- Enroll in courses
- Access learning materials

- Attend assessments
- View marks/results
- Track course progress
- Receive/download certificate

Administrator:

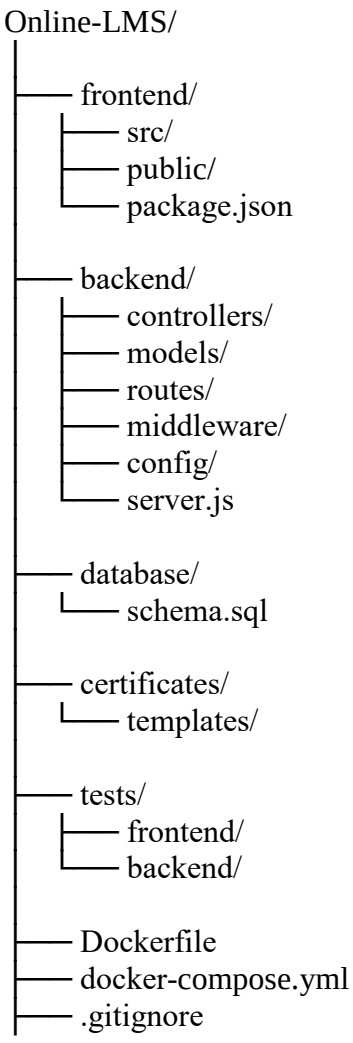
- Manage users
- Manage courses
- Monitor platform activities
- Manage certificates and assessments

Expected Outcomes

The system should provide a centralized e-learning platform where instructors can manage courses and students can learn, complete assessments, monitor their progress, and obtain certificates.

2. Project Structure Design

Proposed Folder Structure



```
├── .env
└── README.md
```

Purpose of Major Folders

3. Git Repository Initialization

Git is used for **version control and project management**. The assignment requires the project repository to be initialized and the essential Git files to be explained. CSA10-CO3 AT1 Git & Docker Tech…

Commands

```
mkdir Online-LMS
cd Online-LMS

git init

git add .
git commit -m "Initial commit"

git branch -M main

git remote add origin <GitHub-Repository-URL>

git push -u origin main
```

Important Git Files

.git/
Stores Git repository information, commit history, branches, and configuration.

.gitignore
Specifies files that should not be tracked.

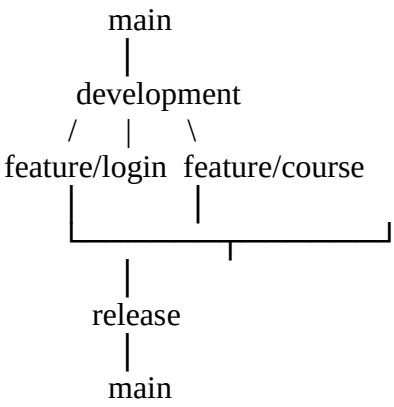
Example:

```
node_modules/
.env
*.log
```

README.md
Contains project description, installation steps, usage instructions, and other documentation.

4. Git Branching Strategy

Proposed Strategy



Branches

Branch	Purpose
main	Stable production version
development	Integration and testing
feature/login	Login functionality
feature/course	Course management
feature/assessment	Assessment functionality
feature/certificate	Certificate generation
release	Final testing before production
hotfix	Emergency bug fixes

Justification

This branching strategy separates stable code from development work. Developers can independently work on features without affecting the main application. After testing, completed features can be merged into the development branch and finally into the main branch.

5. Version Control Workflow

Git Workflow

Create Feature
↓
Create Branch
↓
Write Code
↓
git add
↓
git commit

↓
Push Branch
↓
Pull Request
↓
Code Review
↓
Merge Development
↓
Testing
↓
Merge Main

6. Commit History Analysis

Sample Commit History

Initial commit
Add user authentication
Add instructor course managemen
Add student enrollment module
Add assessment module
Add progress tracking
Add certificate generation
Add Docker configuration
Fix authentication bug
Resolve merge conflict
Update documentation

Good Commit Message Practices

Commit messages should:

- Be short and meaningful.
- Describe one major change.
- Use action-oriented words.
- Avoid vague messages such as "changes" or "update".

Example

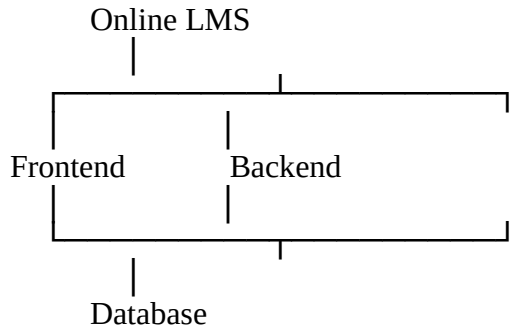
git commit -m "Add student enrollment functionality"
git commit -m "Implement course progress tracking"
git commit -m "Add certificate generation"

A clear commit history makes it easier to understand project development and maintain previous versions. The assignment also evaluates whether commit messages contribute to project maintenance.

7. Docker Environment Design

Docker is suitable for the LMS because the application contains multiple components such as the frontend, backend, and database.

Proposed Architecture



Containers

Container	Purpose
Frontend	Runs LMS user interface
Backend	Runs APIs and application logic
Database	Stores users, courses, enrollments, assessments and progress

Docker provides a consistent environment for development, testing, and deployment. This matches the assignment requirement to identify application components that require containerization.

8. Dockerfile Development

Sample Dockerfile

```
FROM node:20

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 5000

CMD ["npm", "start"]
```

Explanation

Instruction	Purpose
FROM	Selects the base Node.js image
WORKDIR	Sets the application working directory
COPY	Copies project files into the container
RUN npm install	Installs dependencies
EXPOSE	Specifies application port
CMD	Starts the application

The assignment specifically requires the Dockerfile instructions and their purposes to be explained.

9. Docker Compose Design

docker-compose.yml

```
services:

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      DB_HOST: database
      DB_NAME: lms
      DB_USER: root
      DB_PASSWORD: password
    depends_on:
      - database

  database:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: lms
    ports:
```

```
    - "3306:3306"
  volumes:
    - lms_data:/var/lib/mysql

volumes:
  lms_data:
```

Services

- **Frontend:** Provides LMS interface.
- **Backend:** Handles APIs and business logic.
- **Database:** Stores application data.

Networking

Docker Compose creates a network that allows containers to communicate using their service names.

Volume

lms_data stores database data so that the data remains available even if the database container is recreated.

Dependencies

The backend depends on the database, while the frontend depends on the backend.

The assignment specifically asks for services, networking, volumes, environment variables, and dependencies in the Compose configuration. CSA10-CO3 AT1 Git & Docker Tech…

10. Container Deployment and Testing

Build and Start Containers

```
docker compose build
docker compose up
```

Or run in background:

```
docker compose up -d
```

Check Running Containers

```
docker ps
```

Stop Containers

```
docker compose down
```

Testing Procedure

1. Start Docker containers.
2. Open the LMS frontend.
3. Register/login as a student.
4. Browse available courses.
5. Enroll in a course.
6. Access learning materials.
7. Attend an assessment.
8. Check assessment result.
9. Verify progress tracking.
10. Complete the course and verify certificate generation.
11. Check backend and database connectivity.

Expected Result

The LMS should successfully run through Docker containers, with the frontend communicating with the backend and the backend communicating with the database.

The assignment expects evidence such as screenshots showing successful Docker execution and testing. CSA10-CO3 AT1 Git & Docker Tech…

11. Benefits of Git and Docker

Git Benefits

- Provides version control.
- Maintains complete project history.
- Supports branching and merging.
- Enables collaborative development.
- Helps recover previous versions.
- Makes project maintenance easier.

Docker Benefits

- Provides a consistent development environment.
- Avoids dependency-related problems.
- Makes deployment easier.
- Improves portability.
- Supports scalability.
- Separates application components into containers.

Combined Benefit

Git manages the **source code and version history**, while Docker manages the **application environment and deployment**.

Therefore:

Git → Code + Version Control
↓
Docker → Environment + Deployment
↓
Reliable LMS

These benefits directly correspond to the assignment's required discussion of collaboration, version management, portability, deployment, scalability, and maintainability.

12. Challenges and Improvements

Challenges

1. Managing multiple Git branches.
2. Resolving merge conflicts.
3. Maintaining consistent dependencies.
4. Connecting frontend, backend, and database containers.
5. Managing environment variables securely.
6. Maintaining database persistence.
7. Testing communication between containers.

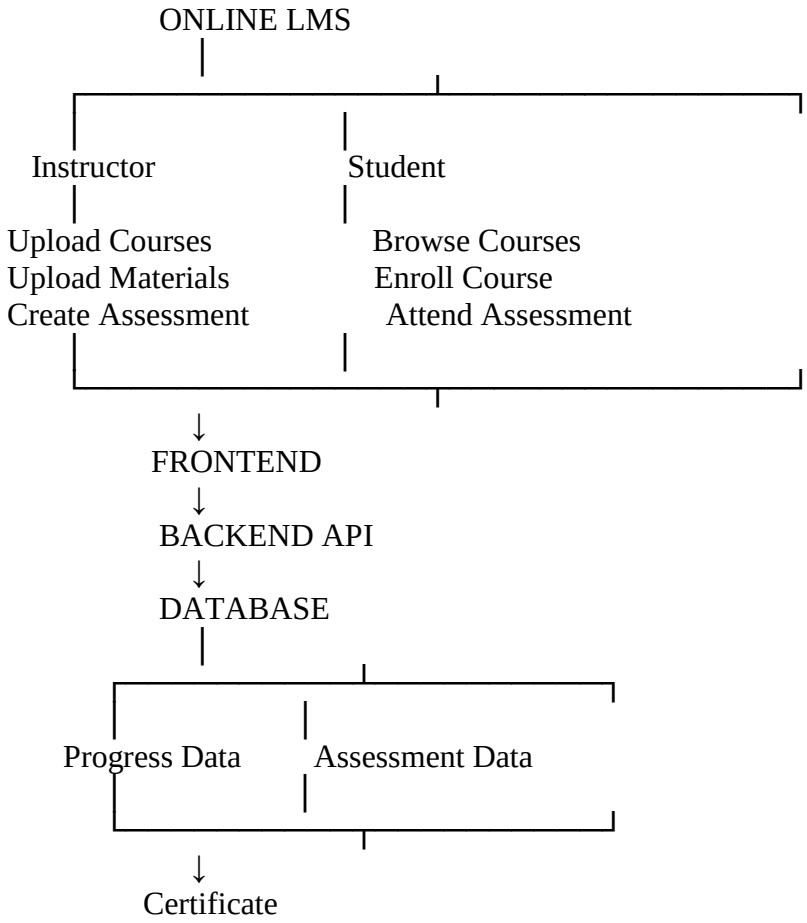
Improvements

- Use GitHub Pull Requests for code review.
- Use automated testing before merging.
- Add CI/CD using GitHub Actions.
- Use Docker health checks.
- Use separate development and production configurations.
- Add database backup mechanisms.
- Implement secure authentication.
- Add monitoring and logging.
- Use HTTPS for secure communication.

Future Enhancements

- Live online classes.
- Video lecture support.
- Discussion forums.
- AI-based course recommendations.
- Personalized learning paths.
- Advanced analytics dashboard.
- Mobile application.
- Automated certificate verification.
- AI-based assessment generation.

Architecture Diagram



Final Result

The Online Course Learning Management System was successfully designed using Git for version control and Docker for containerization. The proposed system supports instructor course management, student enrollment, assessments, progress tracking, and certificate generation. Git provides structured collaboration and version management, while Docker provides a portable and consistent deployment environment.